Git Version control

Introduction

Initial Assessment

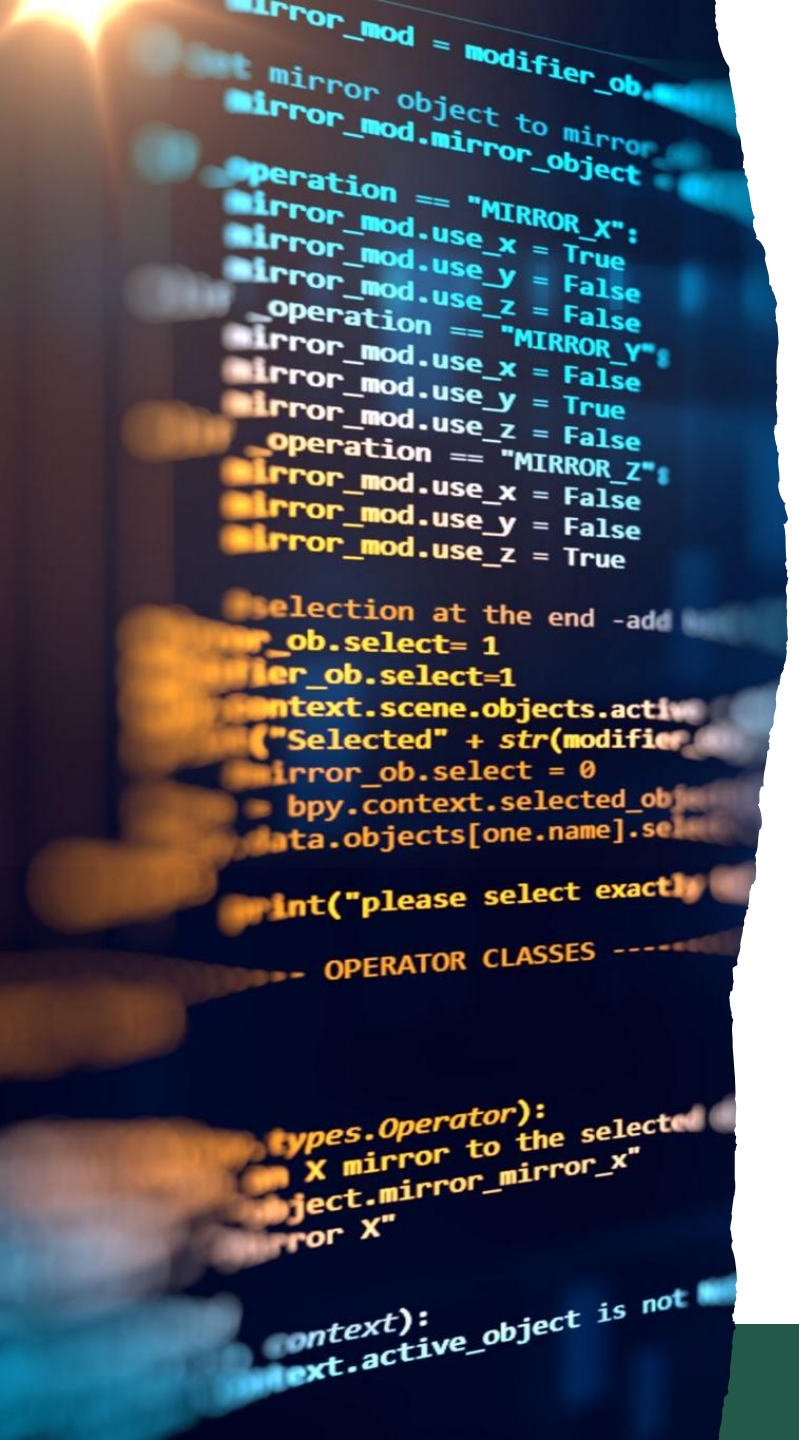
User Interface

What is UI?

Visual Part of computer application or operating system through which a user interacts with a computer or software.

This can include display screens, keyboard, a mouse and the appearance of a desktop.

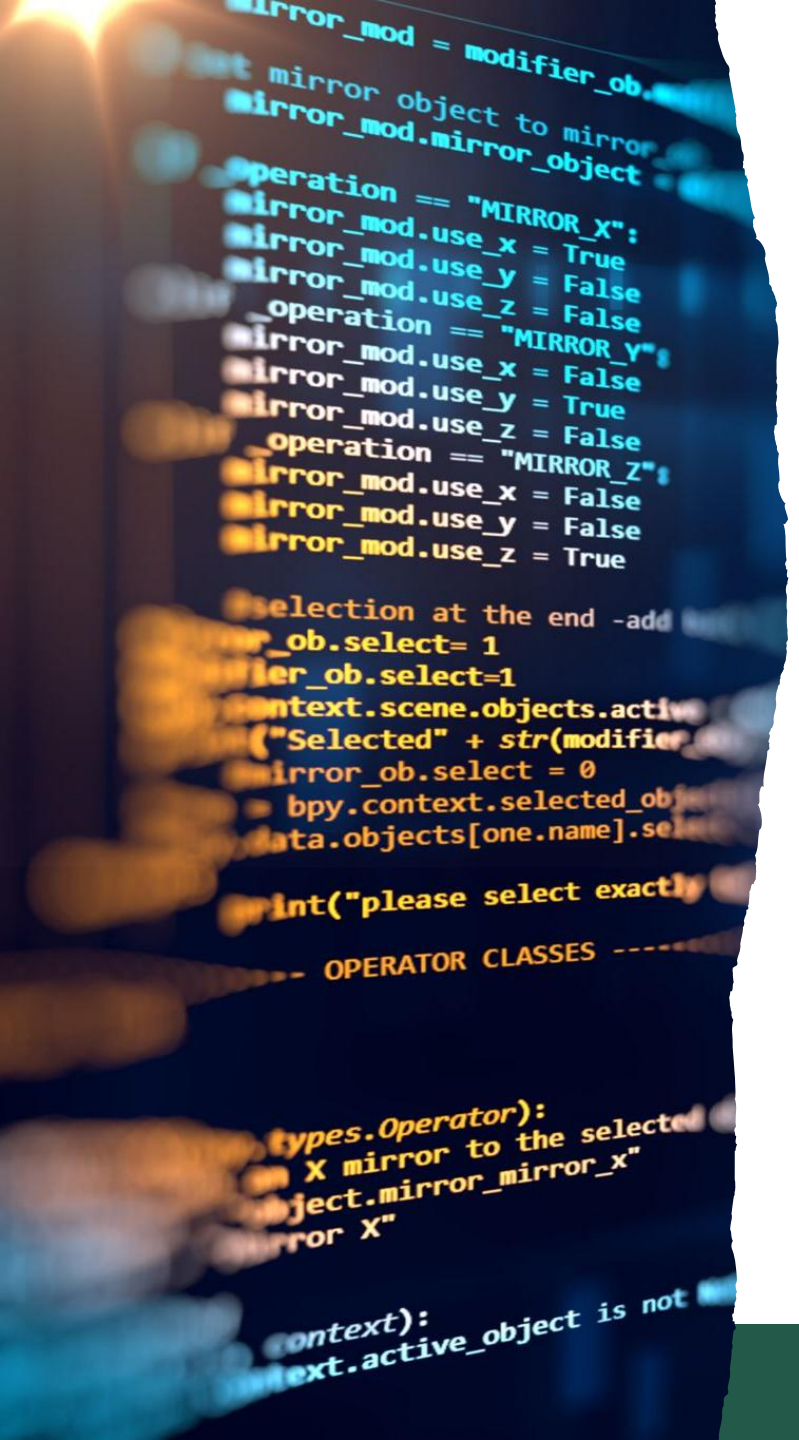
It is also the way of through which a user interacts with an application or website



Types of Interface

There are three types of Interface

- Command Line Interface(CLI): the user must know the machine or program specific instructions or code.
- Menu: user chooses the command from lists displayed menu’s on screen.
- Graphical User Interface(GUI): User gives commands by selecting and clicking on icon displayed on screen.



Command Line Interface(CLI)

Applications:

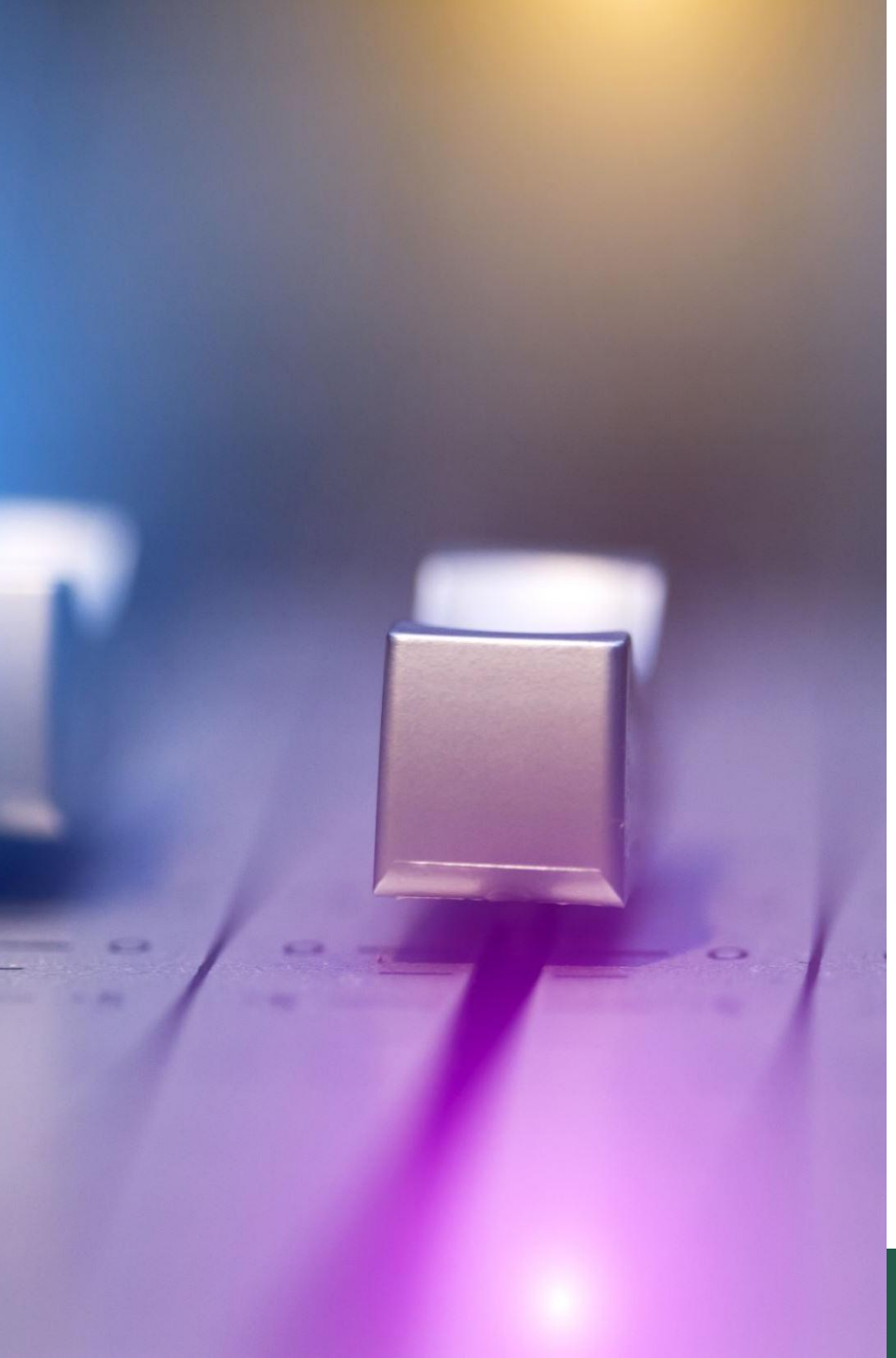
- System Administration
- Scientific Application
- Engineering Application

Advantages:

- Good for expert users, who can access command.
- A CLI requires less memory to use in comparison to other interface.
- A CLI don’t require window and a low-resolution monitor can be used.

Disadvantages:

- Command have to be type precisely. If there is spelling error, the command will fail.
- Not very good for a novice user(beginner, engineer, new in this)

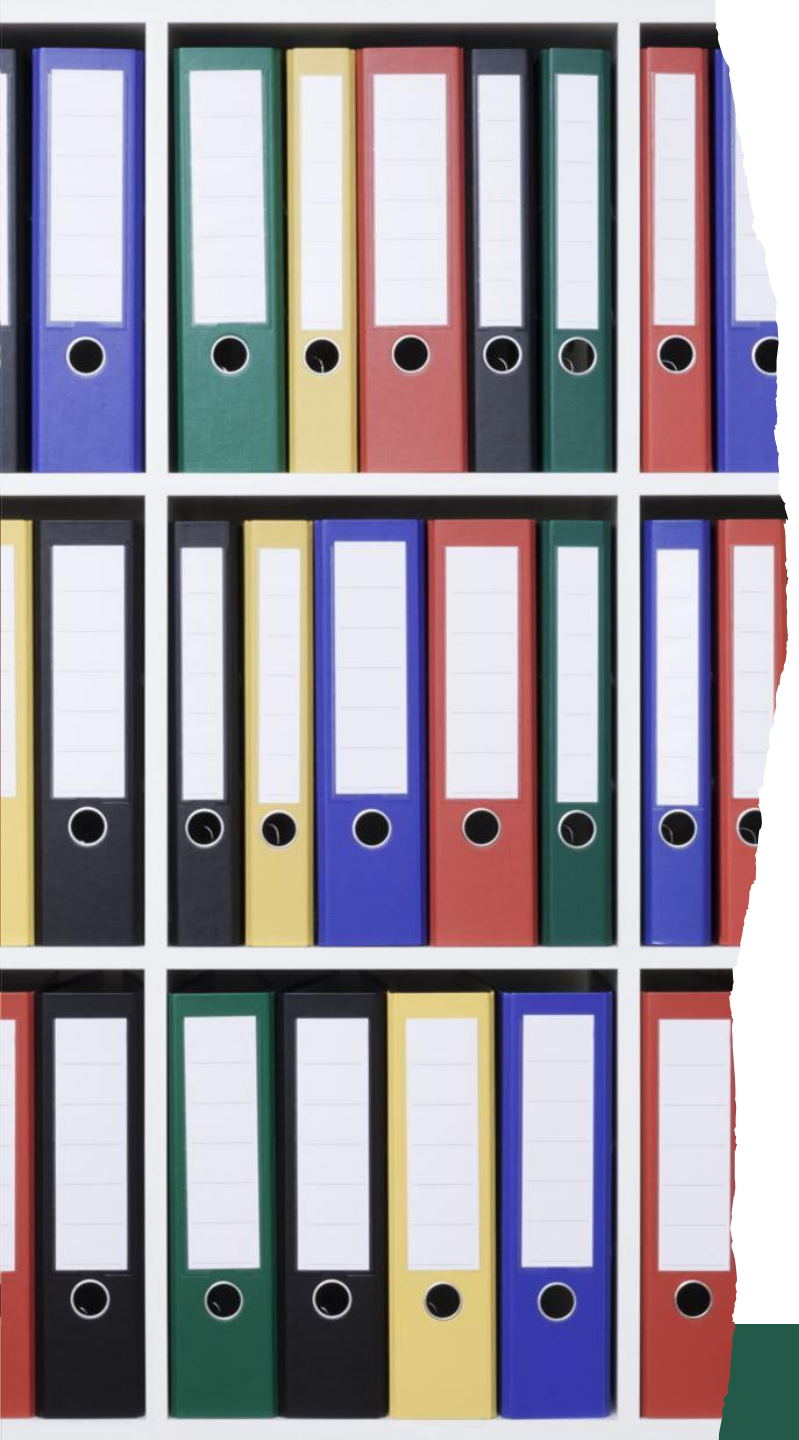


Menu Driven Interface

You are the presented with a menu, you can make a choice and then the next menu appears on the screen. You make another choice and so on.

Application:

- iPod and Mobile
- Cash Withdrawal Machines
- Video Recorder and household device



Advantages and Disadvantages of Menu Driven Interface

Advantages:

- No need to learn complex command language.
- Easier for a novice to learn/use.
- Ideal when there a limited number of options.

Disadvantages:

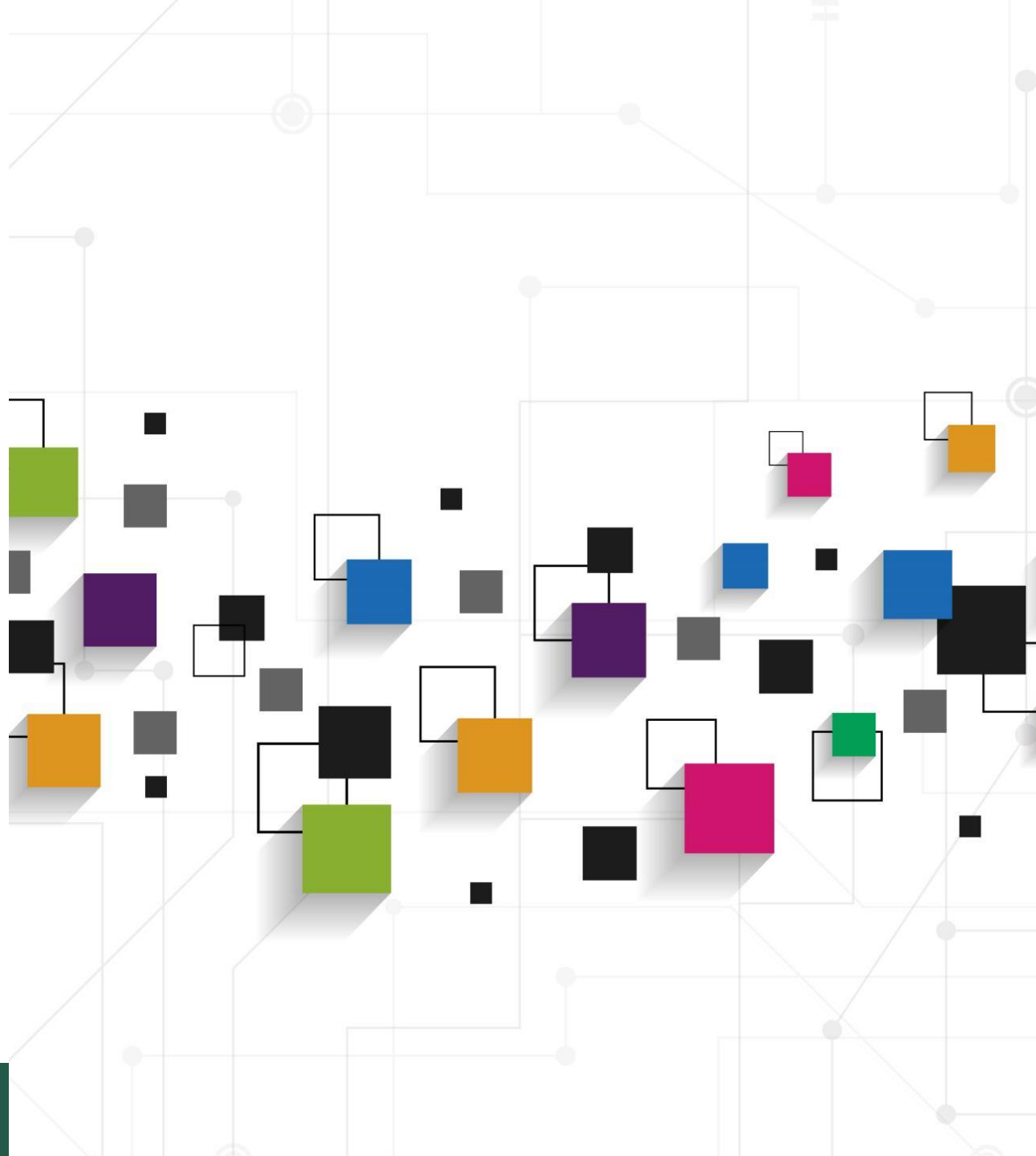
- It can be irritated if there are too many menu screens to work through-user get annoyed or bored if it takes too long.

Graphical User Interface(GUI)

GUI allows user to interact with electronic devices through graphical icons and visual indicators such as secondary notation, instead of text-based user interface typed command labels or text navigation.

Applications:

- GUI used by most commercially popular computer operating systems and software programs today.
- It allows to user that manipulate elements on the screen using a mouse, or even a finger.



Advantages and Disadvantages of GUI

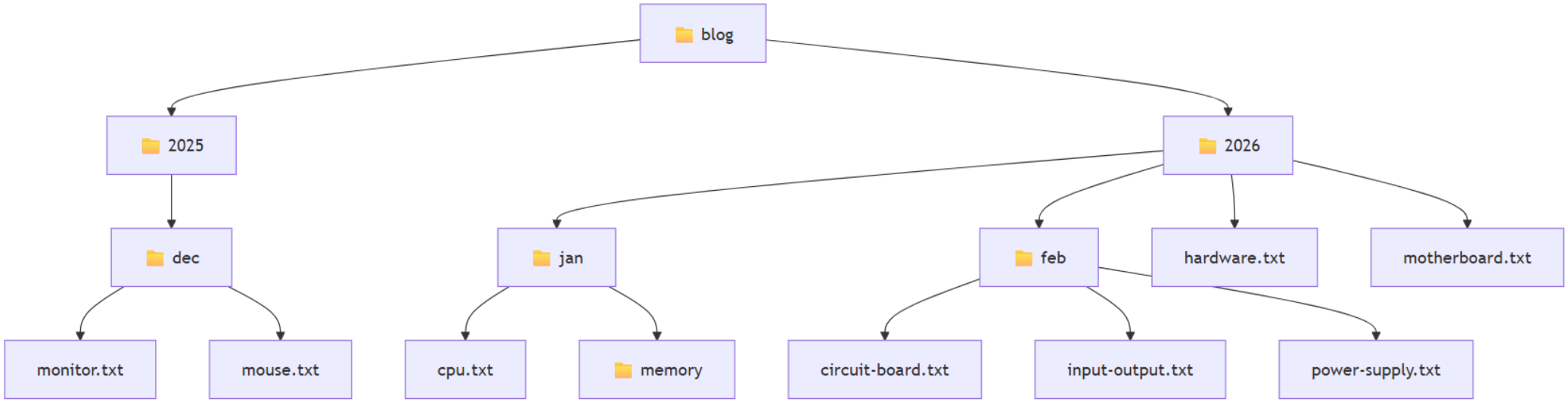
Advantages:

- Less expert knowledge is required to use it.
- Easier to navigate.

Disadvantages:

- Typically decreased options.
- Less customisable

File System


blog
2025
2026
dec
jan
feb
hardware.txt
motherboard.txt
monitor.txt
mouse.txt
cpu.txt
memory
circuit-board.txt
input-output.txt
power-supply.txt

Basic commands for Windows Command Prompt

dir lists all files and directories in the working directory.

cd switches you into the directory you specify.

mkdir creates a new directory in the working directory.

notepad creates a new file inside the working directory

Assessment

- Activity time

Basic commands for Linux

pwd outputs the name of the current working directory.

ls lists all files and directories in the working directory.

cd switches you into the directory you specify.

mkdir creates a new directory in the working directory.

touch creates a new file inside the working directory

What is Git?

- **Git** is a **version control system.**

  It helps you:
- Track changes in your code
- Go back to older versions
- Work with other people without breaking things
- Think of Git as **“undo + history + teamwork” for code.**

Why Do We Need Git?

Without Git:

You might overwrite files

You lose old versions

Teamwork becomes messy

With Git:

Every change is saved

Mistakes are reversible

Multiple people can work together smoothly

Git vs GitHub (Very Important!)

Git

- A tool
- Runs on your computer
- Tracks changes

GitHub

- A website
- Stores Git projects online
- Helps people collaborate

Git = engine
GitHub = parking + sharing place

Basic Git Concepts

- **Repository (Repo)**
  A project folder tracked by Git
- **Commit**
  A saved checkpoint of your work
- **Branch**
  A separate version of your code
- **Main / Master**
  The main branch of the project

- Download: https://git-scm.com/install/windows

git version

- git version

```
H:\>git version
git version 2.52.0.windows.1
```

git help

```
H:\>git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
           [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
           [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
           [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
           [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
           <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
   clone      Clone a repository into a new directory
   init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add        Add file contents to the index
   mv         Move or rename a file, a directory, or a symlink
   restore    Restore working tree files
   rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
   bisect     Use binary search to find the commit that introduced a bug
   diff       Show changes between commits, commit and working tree, etc
   grep       Print lines matching a pattern
   log        Show commit logs
   show       Show various types of objects
   status     Show the working tree status
```

Git Setup

- Open the command prompt and execute the following git commands to setup you're user profile

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

C:\sw-setup\workspace\git-examples>git config --global user.name "Savi Praveen"

C:\sw-setup\workspace\git-examples>git config --list
core.symlinks=false
core.autocrlf=true
core.fscache=true
color.interactive=true
color.ui=auto
help.format=html
diff.astextplain.textconv=astextplain
rebase.autosquash=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
credential.helper=helper-selector
user.name=Savi Praveen

C:\sw-setup\workspace\git-examples>
```

Email id

```
C:\sw-setup\workspace\git-examples>git config --global user.email "savithiri.praveenku@oaklands.ac.uk"

C:\sw-setup\workspace\git-examples>git config --list
core.symlinks=false
core.autocrlf=true
core.fscache=true
color.interactive=true
color.ui=auto
help.format=html
diff.astextplain.textconv=astextplain
rebase.autosquash=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
credential.helper=helper-selector
user.name=Savi Praveen
user.email=savithiri.praveenku@oaklands.ac.uk
```

Setup a project in your workstation

Create a new git project in your workstation

Open command prompt and navigate to the respective folder location in your workstation

Git init

The word `init` means *initialize* . The command sets up all the tools Git needs to begin tracking changes made to the project

```
C:\sw-setup\workspace\git-examples>git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in C:/sw-setup/workspace/git-examples/.git/
```

Git Workflow

- A Git project can be thought of as having three parts:
  1. A *Working Directory* : where you'll be doing all the work: creating, editing, deleting and organizing files
  2. A *Staging Area* : where you'll list changes you make to the working directory
  3. A *Repository* : where Git permanently stores those changes as different *versions* of the project
- The Git workflow consists of editing files in the working directory, adding files to the staging area, and saving changes to a Git repository.

Basic Git Workflow

1

Working Directory

Make changes to files:

- Additions
- Deletions
- Modifications

git add

2

Staging Area

Files are ready for commit

git commit

3

Repository

Changes are saved to repository as a commit

Git status

show modified files in working directory,
staged for your next commit

```
C:\sw-setup\workspace\git-examples>git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Create a new file in your working directory

```
C:\sw-setup\workspace\git-examples>notepad todolist.txt
```

```
Hello World! We are going to make these in the to do list:
```

Verify the newly created file by using

- "dir" command:

```
C:\sw-setup\workspace\git-examples>dir
 Volume in drive C is Windows
 Volume Serial Number is CC52-7225

 Directory of C:\sw-setup\workspace\git-examples

02/02/2026  18:59    <DIR>          .
02/02/2026  18:13    <DIR>          ..
02/02/2026  19:00                60 todolist.txt
               1 File(s)             60 bytes
               2 Dir(s)  325,829,644,288 bytes free
```

Git status: check newly created file

```
C:\sw-setup\workspace\git-examples>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        todolist.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Second file

```
C:\sw-setup\workspace\git-examples>notepad todolist-2.txt
```

```
You're in the second file
```

```
C:\sw-setup\workspace\git-examples>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        todolist-2.txt
        todolist.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Git add

- **git add [file]**

add a file as it looks now to your next commit (stage)

```
C:\sw-setup\workspace\git-examples>git add todolist.txt

C:\sw-setup\workspace\git-examples>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   todolist.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        todolist-2.txt
```

Open notepad and make some changes

- Step 1: Open the file

```
C:\sw-setup\workspace\git-examples>notepad todolist.txt
```

Step 2: Edit and save the file

```
Hello World! We are going to make these in the to do list:
01. Making my changes
```

Step 3: verify the status of the file (cont..)

Step 3: verify the status of the file(cont..)

```
C:\sw-setup\workspace\git-examples>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   todolist.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   todolist.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        todolist-2.txt
```

git diff

- **git diff**
- diff of what is changed but **not staged**

```
C:\sw-setup\workspace\git-examples>git diff todolist.txt
diff --git a/todolist.txt b/todolist.txt
index db1ebc9..f59e601 100644
--- a/todolist.txt
+++ b/todolist.txt
@@ -1 +1,2 @@
 Hello World! We are going to make these in the to do list:
+01. Making my changes
```

git commit

- A *commit* is the last step in our Git workflow. A commit permanently stores changes from the staging area inside the repository.
  git commit -m "[descriptive message]"

Standard Conventions for Commit Messages:

- Must be in quotation marks
- Written in the present tense
- Should be brief (50 characters or less) when using -m

```
C:\sw-setup\workspace\git-examples>git commit -m "initial commit"
[master (root-commit) 575517b] initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 todolist.txt
```

Git log

- you'll need to refer back to an earlier version of a project. Commits are stored chronologically in the repository and can be viewed with:

```
C:\sw-setup\workspace\git-examples>git log
commit 575517b9599cf08125b9773aa891c13545b1fdb5 (HEAD -> master)
Author: Savi Praveen <savithiri.praveenku@oaklands.ac.uk>
Date:   Mon Feb 2 19:37:22 2026 +0000

    initial commit
```

Revisit

- You have now been introduced to the fundamental Git workflow. You learned a lot! Let’s take a moment to generalize:
- Git is the industry-standard version control system for web developers
- Use Git commands to help keep track of changes made to a project:
  - init creates a new Git repository
  - git status inspects the contents of the working directory and staging area
  - git add adds files from the working directory to the staging area
  - git diff shows the difference between the working directory and the staging
  - area

HOW TO BACKTRACK

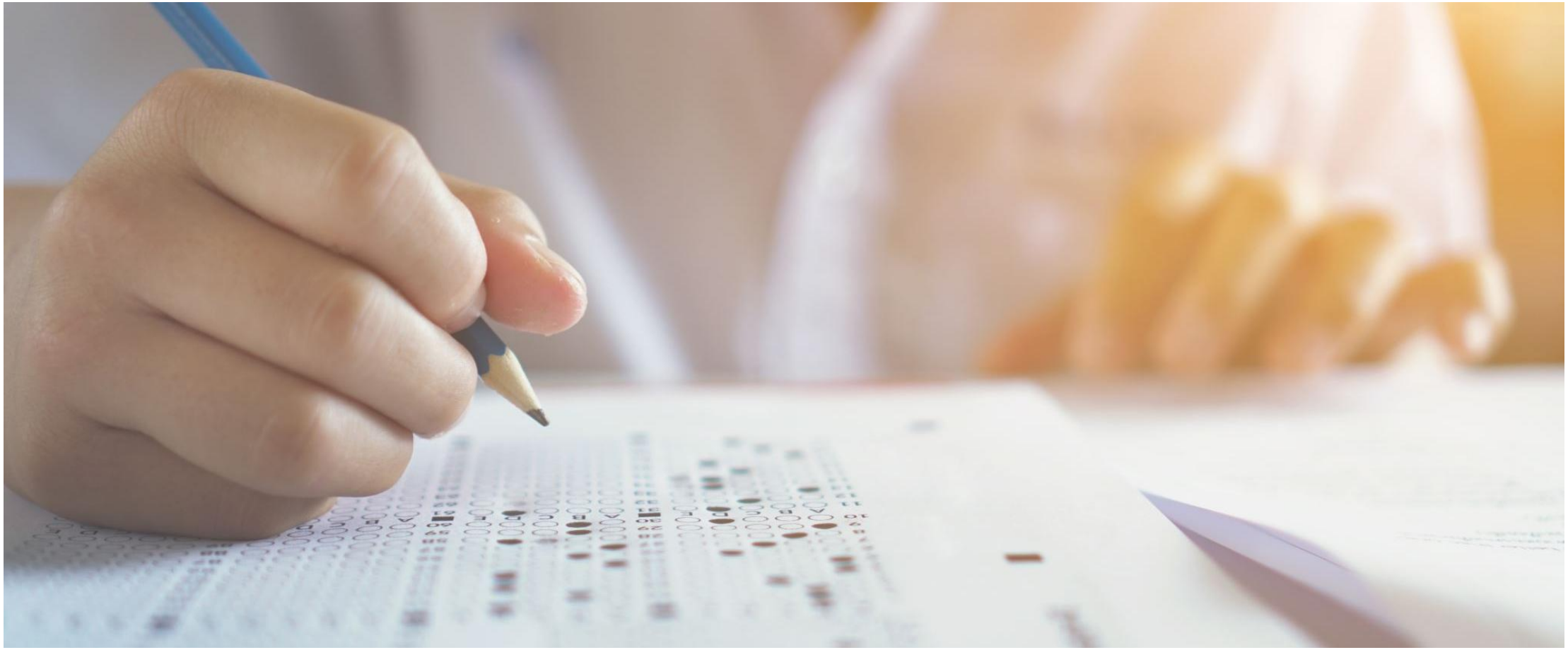
- When working on a Git project, sometimes we make changes that we want to get rid of. Git offers a few eraser-like features that allow us to undo mistakes during project creation. In this lesson, we'll learn some of these features.
- To start out, let's review the basic Git workflow.
- $ mkdir hamlet-prince-of-denmark
- $ cd hamlet-prince-of-denmark
- $ notepad scene1.txt

Ghost:

  - My hour is almost come,
  - When I to sulphurous and tormenting flames
  - Must render up myself.

- $ git add scene-1.txt
- $ git commit -m "my hour"

Learning Check



Github – Online demo

- https://github.com/

What is a Branch?

A **branch** lets you:

- Try new features
- Fix bugs safely
- Not break the main code

Example:

- Main branch = working website
- New branch = experiment zone

Merging Branches

- **Merge** = combining branches
- Example:
- Feature branch is finished
- Merge it into main
- Now main has the new feature

Working with GitHub

Typical teamwork flow:

Push code to GitHub
Others pull the code
Everyone works together
Changes are merged

This is how real companies work

Benefits of Using Git

Tracks history
- Easy collaboration
- Backup of code
- Industry standard
- Saves time & stress

Who Uses Git?

- Software developers
- Data scientists
- Web designers
- Big companies (Google, Microsoft, Netflix)

If you code → you'll use Git.

Final Thoughts

- Git is not scary
- Everyone learns it step by step
- Practice is the key
- **Start small. Commit often.**

Advanced Learning:

- Undoing Changes: git restore, git reset
- Branching: Create & switch branches
- Merging: Fast-forward & recursive merges
- Merge Conflicts: How to resolve conflicts
- Rebasing: Replay commits for linear history

Git and GitHub Tutorial – Version Control for Beginners

https://www.freecodecamp.org/news/git-and-github-for-beginners/